Recitation 13: Reinforcement Learning

Hariharan, Aishwarya

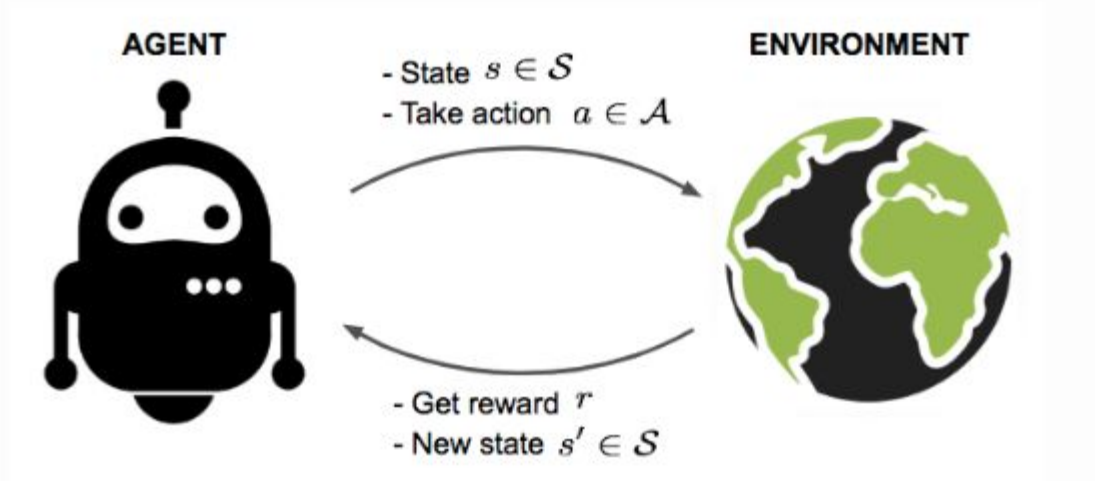
Teaching a pigeon how to read

Reinforcement Learning draws inspiration from:

- Psychology
- Control Theory
- Evolution
- Computer Science (Dynamic Programming)

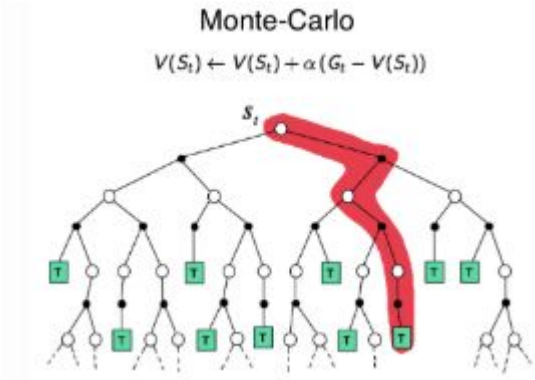
Introduction

- Paradigm of RL is different from Supervised Learning
- A reward function is engineered/learnt that serves as the “objective” to be maximized
- The agent has to learn how to act to maximize the reward.



Let’s learn the dynamics of the environment!

Monte Carlo Methods

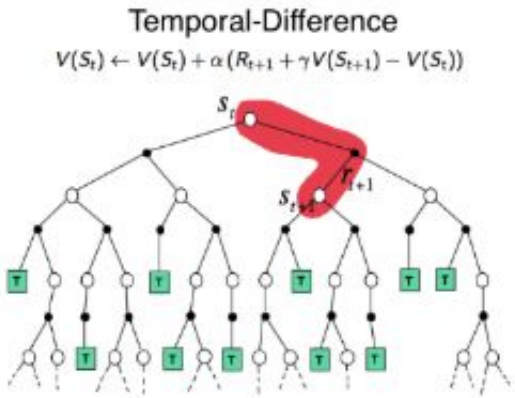


What is a Monte Carlo method ?

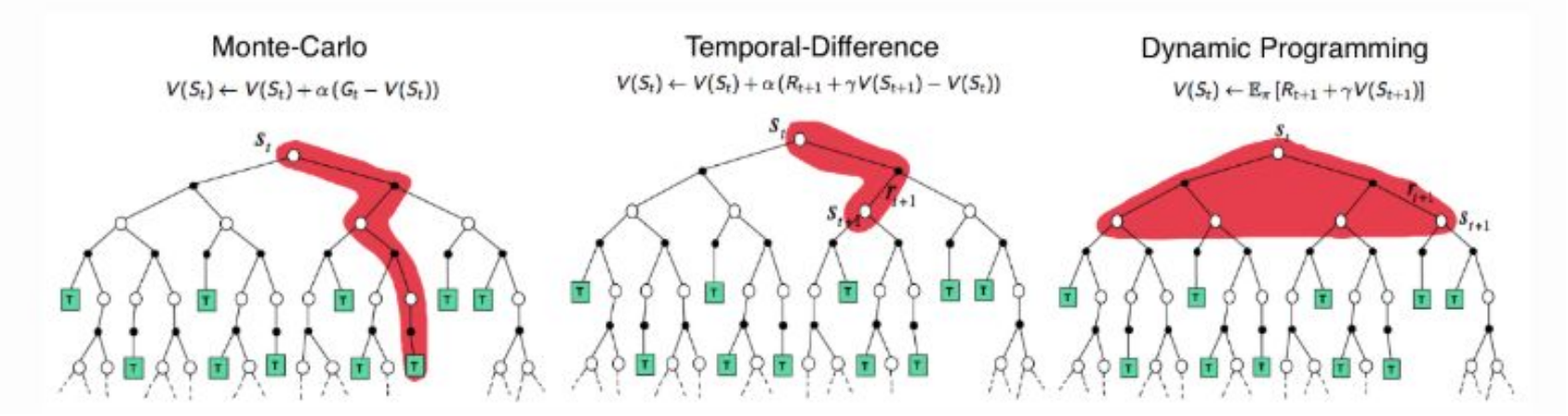
- It is a model-free method
- When the transition probabilities between states are unknown, we need to estimate them.
- Monte Carlo starts with random state initializations and plays out episodes till the end.
- Based on the episodes observed, state transition probabilities are updated

TD learning

- Also a model-free method
- Instead of waiting till the end of the episode for the reward, we can estimate the intermediate rewards at every instant.
- SARSA is on-policy TD learning
- Q Learning is off-policy TD learning



Comparison of Monte-Carlo, TD and DP



Can we borrow from supervised learning? Imitation learning

- Maximize reward by learning to imitate expert actions.

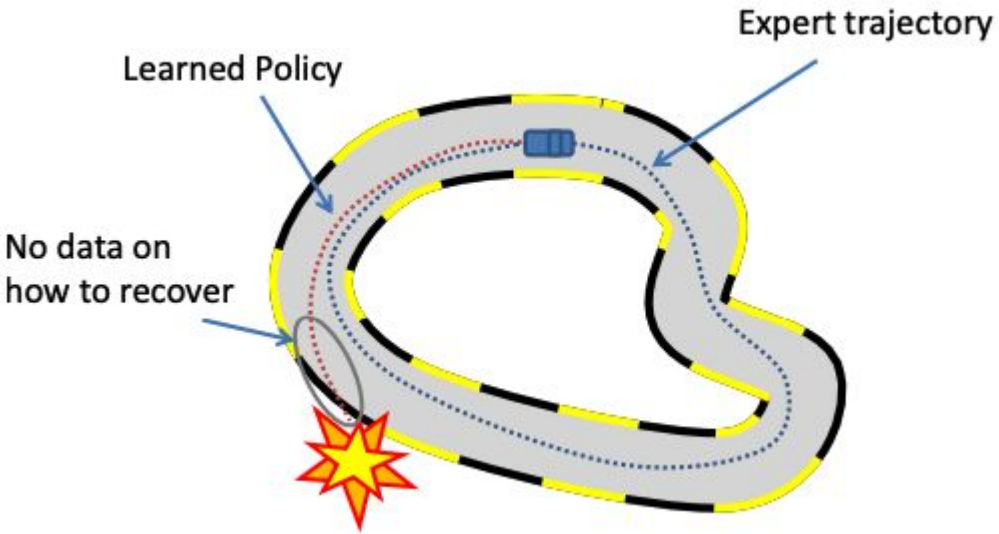
$$\tau = \{ s_1, a_1, s_2, a_2, \ldots, s_T, a_T \}$$

supervised training $s_1 \rightarrow a_1, \ldots, s_T \rightarrow a_T$

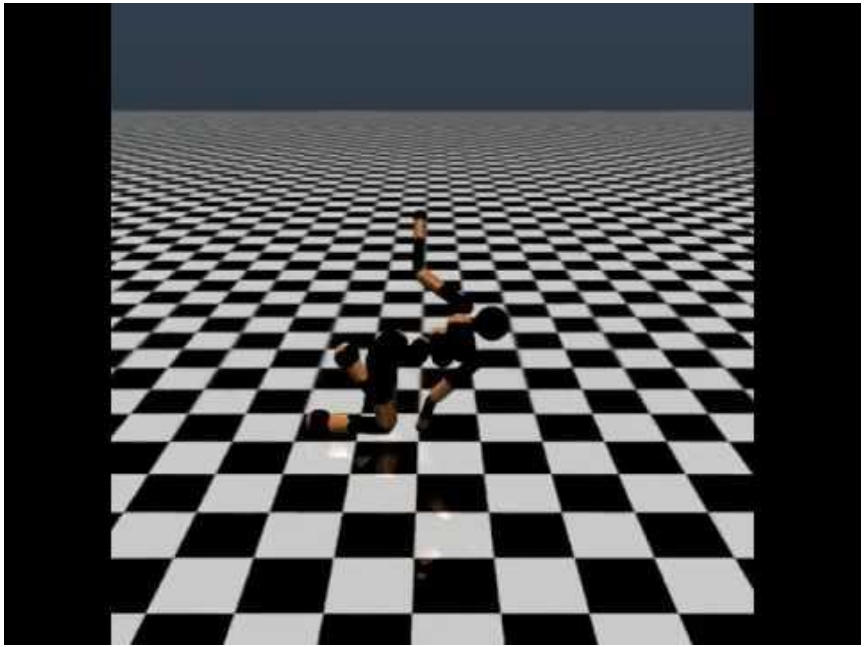
Behaviour cloning

- It is a type of Imitation Learning
- Clone the behaviour of experts.
- Train in a supervised fashion

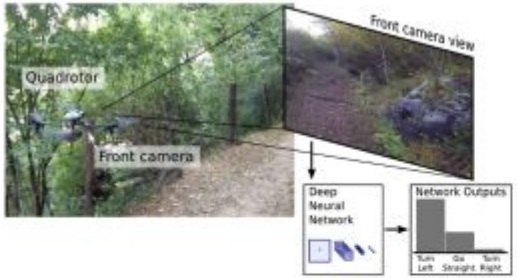
Where it fails ?

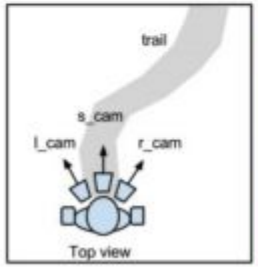


Compounding errors

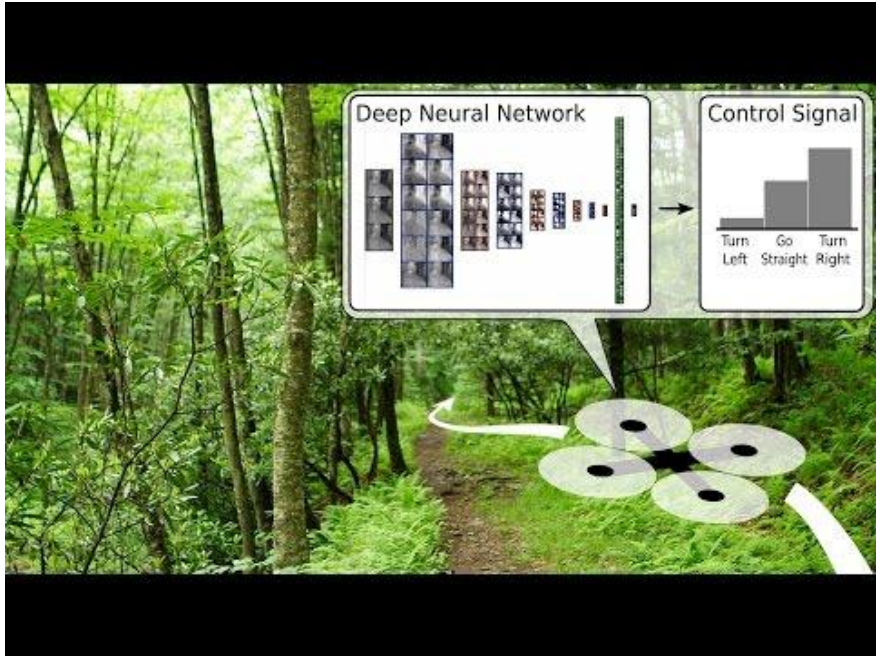


Solution - Dataset AGGregation (DAGGER)





Let’s fly a drone!



GAIL (Generative Adversarial Imitation Learning)

- Generator predicts the action and the next state
- Discriminator classifies if tuple of action and next state was observed by a student or an expert.

Algorithm 1 Generative adversarial imitation learning

1: **Input:** Expert trajectories $\tau_E \sim \pi_E$, initial policy and discriminator parameters θ_0, w_0
2: **for** $i = 0, 1, 2, \ldots$ **do**
3: Sample trajectories $\tau_i \sim \pi_{\theta_i}$
4: Update the discriminator parameters from w_i to w_{i+1} with the gradient

$$\hat{\mathbb{E}}_{\tau_i}[\nabla_w \log(D_w(s,a))] + \hat{\mathbb{E}}_{\tau_E}[\nabla_w \log(1 - D_w(s,a))] \tag{17}$$

5: Take a policy step from θ_i to θ_{i+1}, using the TRPO rule with cost function $\log(D_{w_{i+1}}(s,a))$. Specifically, take a KL-constrained natural gradient step with

$$\begin{aligned} &\hat{\mathbb{E}}_{\tau_i}[\nabla_\theta \log \pi_\theta(a|s) Q(s,a)] - \lambda \nabla_\theta H(\pi_\theta), \\ &\text{where } Q(\bar{s},\bar{a}) = \hat{\mathbb{E}}_{\tau_i}[\log(D_{w_{i+1}}(s,a)) \mid s_0 = \bar{s}, a_0 = \bar{a}] \end{aligned} \tag{18}$$

6: **end for**

GAIL - Landing the rover

Value Function, Q Function and Bellman Equation

What is a value function?

- Determines how valuable a given state is, for the agent.
- The value function depends on the policy using which the agent performs actions
- The value at a particular state using a policy $\boldsymbol{\pi}$ is given by:

$$V^{\pi}(s) = \mathbb{E}[\sum_{i=1}^{T} \gamma^{i-1} r_i] \quad \forall s \in \mathbb{S}$$

- Among all value-functions, there exists an optimal value function whose value is greater that other functions for all states. The optimal policy $\boldsymbol{\pi}^*$ corresponds to the optimal value

$$V^*(s) = \max_{\pi} V^{\pi}(s) \quad \forall s \in \mathbb{S} \qquad \pi^* = \arg\max_{\pi} V^{\pi}(s) \quad \forall s \in \mathbb{S}$$

Value Function, Q Function and Bellman Equation

What is the Q-value function?

- Determines how valuable taking an action a is, from a given state s
- V*(s) can be obtained by finding the maximum over all possible Q*(s,a) values
- The Q*(s, a) is equal to the summation of immediate reward after performing action a while in state s and the discounted expected future reward after transition to a next state s'.
- If we know the optimal Q-function we can extract the optimal policy by choosing the right action that maximises Q for a state s

Value Function, Q Function and Bellman Equation

The Bellman Equation:

$$Q^*(s,a) = R(s,a) + \gamma\, \mathbb{E}_{s'}[V^*(s')]$$

$$Q^*(s,a) = R(s,a) + \gamma \sum_{s' \in \mathbb{S}} p(s'|s,a) V^*(s')$$

Since,

$$V^*(S) = \max_a Q^*(s,a)$$

$$V^*(S) = \max_a \left[R(s,a) + \gamma \sum_{s' \in \mathbb{S}} p(s'|s,a) V^*(s') \right]$$

Value Iteration

- Computed the optimal state value function by improving the value of V(s) iteratively from a random start value
- Repeatedly updates Q(s,a) and V(s) until convergence and it is guaranteed to converge to optimal values.

```
Initialize V(s) to arbitrary values
Repeat
    For all s ∈ S
        For all a ∈ A
            Q(s,a) ← E[r|s,a] + γ Σ_{s'∈S} P(s'|s,a)V(s')
        V(s) ← max_a Q(s,a)
Until V(s) converge
```

Policy Iteration

- In value iteration, since the agent is optimising for the optimal policy, it might converge before value function.
- In Policy iteration, instead of repeatedly improving the value function, the policy is redefined at each step and the value is computed until convergence.

```
Initialize a policy π' arbitrarily
Repeat
    π ← π'
    Compute the values using π by
        solving the linear equations
            V^π(s) = E[r|s, π(s)] + γ Σ_{s'∈S} P(s'|s, π(s)) V^π(s')
    Improve the policy at each state
        π'(s) ← arg max_a (E[r|s, a] + γ Σ_{s'∈S} P(s'|s, a) V^π(s'))
Until π = π'
```

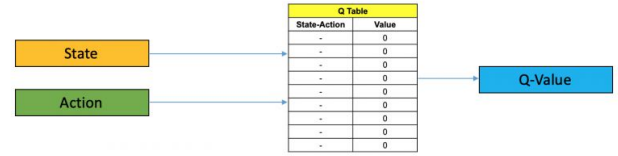
Q Learning

- Policy and Value iteration can be used when the agent has prior knowledge about the effects of its actions and the environment (offline planning)
- What if the agent only knows a set of possible states and actions and can observe the environment current state?
    - The agent must actively learn through its interactions with the environment
- Q-Learning a model-free learning algorithm that does not assume anything about the state-transition or rewards
- Q-learning tries to approximate the state-action pairs from the samples of Q(s,a) that were observed during the interaction with the environment.

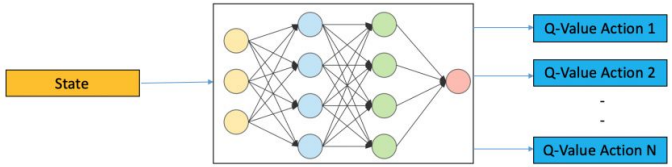
Deep Q Learning

Why deep Q learning?

- If the number of actions and states in an environment are huge, tabulation becomes cumbersome due to both memory and time constraints
- Neural models can be used to approximate Q-values instead
- The state is given as the input and the Q-value of all possible actions is generated as the output



Q Learning

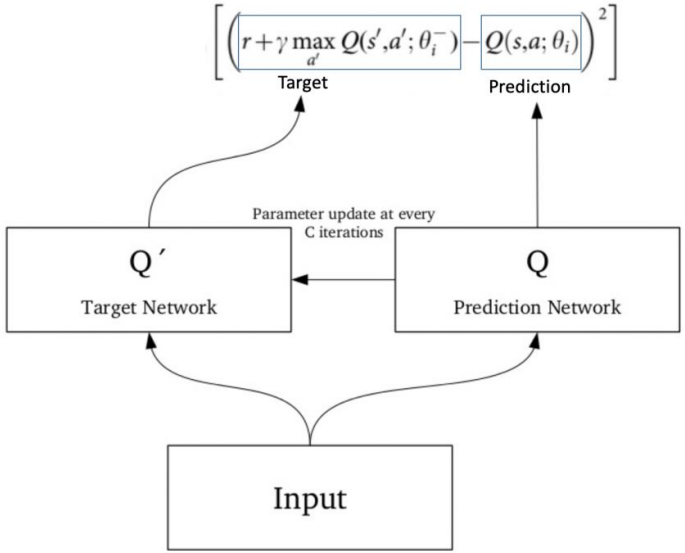


Deep Q Learning

Deep Q Learning

What happens in DQNs?

- The past experiences are stored in a memory buffer and the next action is predicted by the Q network
- Loss is calculated as the mean squared error of the predicted Q value and a target Q value (Q*)
- For calculating the target Q value we can use a separate target network that can reduce divergence
- Target network has the same architecture as the Q-value prediction network but with the parameters frozen
- For every x iterations we copy the parameters from the prediction network to the target network
- This stabilizes training and reduces variability



Deep Q Learning

DQN steps summarized:

- Collect transitions from the environment to train the DQN.
- Select an action using the Epsilon-Greedy policy, i.e., select a random action versus maximum Q value action with a probability epsilon.
- Perform the action in a state s and move to a new state s' and store this transition in the memory buffer <s,a,r,s'>
- Sample a batch of transitions from the replay buffer and calculate the loss
- Perform a gradient descent with respect to the actual network parameters to minimise the loss
- After every x steps copy actual network weights to the target network weights and repeat this for M episodes